

LLM OBSERVABILITY & COST MANAGEMENT

Complete Curated Study Notes
Pablo Deshon · Production-Ready Strategies · Hands-On

WHAT YOU WILL LEARN Save your organisation thousands of dollars · Master LangFuse observability · Implement prompt optimisation, semantic caching & smart model routing · Build production-grade LLM pipelines with full traceability.

SECTION 1 — THE BUSINESS CASE: WHY OBSERVABILITY = MONEY

What Most Teams Discover Too Late

- Runaway prompt:** A single bad prompt can cost \$10,000 in one afternoon.
- Token spike:** Token usage can spike 300% with nobody knowing why.
- Silent failures:** Users complain about slow responses but you can't find the bottleneck.
- Hallucination:** RAG pipelines retrieve garbage and the LLM hallucinates confidently.
- Budget burn:** One developer's retry loop can drain an entire quarterly budget in 2 weeks.

Real Cost Comparison: With vs Without Observability

Category	Without Observability	With Observability	Saving
Monthly LLM Spend	\$35,000+	\$15,000	\$20,000 saved
Wasted Retries	\$5,000+	~\$0	\$5,000 saved
Debug Time / Week	20+ hours	~2 hours	18 hrs saved
Incident Response	Days	Same day	Critical

Risk Matrix Without Observability

Risk	Impact	Probability
Token Spike	HIGH (up to \$10K+)	HIGH
Silent Failures	MEDIUM	HIGH
Performance Degradation	MEDIUM	HIGH
Compliance Violations	CRITICAL	MEDIUM

ROI Formula

Formula Savings = Token Waste + (Debug Time × Rate) + (Incidents Prevented × Cost)
Typical result: 47% token cost reduction + 80% less debugging time + \$0 runaway surprises.

ROI Calculator Item	Value
---------------------	-------

Monthly LLM Spend	\$20,000
Estimated Waste (30%)	\$6,000
Debug Time (10h × \$100/h)	\$4,000/month
Incident Cost (amortised)	\$5,000
Total Preventable Cost	\$15,000 / month
Observability Platform Cost	\$500 – \$2,000/month
ROI (first month)	7× – 30×

SECTION 2 — LLM OBSERVABILITY vs TRADITIONAL MONITORING

Traditional vs LLM Observability

Aspect	Traditional Monitoring	LLM Observability
View of App	Black box (request → response)	Full traceability of every step
Tracks	Latency & HTTP errors	Token flows, prompt quality, cost, hallucination
Cost Visibility	None	Per request / per feature / per user
Debug Speed	Hours	Minutes
Cost Savings	—	50–80% possible

The Three Pillars of LLM Observability

- 1. **TRACES**: End-to-end request flows through every step — what happened from input to output.
- 2. **METRICS**: Token usage, latency, cost-per-request, error rates — quantitative health signals.
- 3. **EVALUATION**: Quality scores and output assessment — is the LLM doing what it should?

What to Measure

Metric	What to Track
Token Usage	Input/output ratio per request
Latency	Per-step breakdown (retrieval, generation, etc.)
Cost	Per request · per feature · per user
Quality	Relevancy score, hallucination rate
Errors	Categorised by type, model, and prompt

SECTION 3 — TOKEN ECONOMICS: WHERE YOUR MONEY GOES

Token Basics

- 1 token ≈ 4 characters in English (NOT characters, NOT words)
- "Hello World" = 4 tokens | 44-character sentence = 10 tokens
- Code tokenises inefficiently — lots of small tokens for syntax characters
- Output tokens cost 2–5× MORE than input tokens
- Verbose responses are expensive · Long system prompts are relatively cheap
- KEY RULE: Ask the model to be concise → always saves money

📌 2025–2026 Model Pricing (per 1M tokens)

Model	Input / 1M	Output / 1M	Notes
GPT-4o	\$2.50	\$10.00	Flagship — balanced
GPT-4o Mini	\$0.15	\$0.60	16× cheaper than GPT-4o
Claude 3.5 Sonnet	\$3.00	\$15.00	Best for complex reasoning
Claude 3.5 Haiku	\$0.25	\$1.25	Best cheap Anthropic model
Gemini 1.5 Pro	\$1.25	\$5.00	Google alternative

🔑 **KEY INSIGHT** Difference between GPT-4o and GPT-4o Mini at 1M requests/month: \$415 vs \$25. That's a 200× price gap! Always choose the cheapest model that can do the job.

📌 RAG + Agentic Pipeline Cost Anatomy

Pipeline Step	Approx Cost (1 query)	Notes
Embeddings (user query)	< \$0.001	Very cheap
Vector Search	< \$0.001	Minimal
Context Assembly	< \$0.001	Cheap
First LLM Call	< \$0.001	Small but scales
Agent Decision + Tool Call	< \$0.001	Adds up with loops
Final Response Generation	< \$0.001	Output-heavy = expensive
TOTAL (1 query)	~\$0.007	Seems cheap...
× 100,000 queries/day	\$700/day	Expensive!
× 30 days	\$21,000/month	Very expensive
× 12 months	\$252,000/year	Critical to optimise

🔑 **This assumes ZERO retries, ZERO errors, ZERO verbose responses!** Any of these will multiply your cost significantly.

📌 Top 5 LLM Cost Drivers

- 📌 **#1 Bloated System Prompts:** Every single request carries the full system prompt payload.
- 📌 **#2 Excessive RAG Context:** More context is NOT always better — retrieve the RIGHT chunks.
- 📌 **#3 Agent Reasoning Loops:** Verbose internal reasoning still costs inference tokens.
- 📌 **#4 Growing Chat History:** Chat context grows linearly → more tokens per turn → more cost.
- 📌 **#5 Wrong Model Selection:** 200× price difference between models — highest leverage point!

📌 Hidden Multipliers — Memorise These

- 🔑 **HIDDEN MULTIPLIERS**
1. System prompts are sent with EVERY request
 2. RAG context can be thousands of tokens per query
 3. Retries = paying twice or more
 4. Chat history grows linearly (each turn adds more tokens)
 5. Wrong model = 200× cost difference

📌 Task → Model Routing Matrix

Task Type	Recommended Model	Why
Classification / Yes-No	Haiku / Mini	Simple → cheapest model

Simple Extraction	Haiku / Mini	No reasoning depth needed
Summarisation	Mini / Mid-tier	Medium complexity
Complex Reasoning	Sonnet / GPT-4o	Needs deeper analysis
Code Generation	Test both tiers	Quality matters — validate output
Creative Writing	Mid / High tier	Quality over cost here

📖 SECTION 4 — LANGFUSE: YOUR OBSERVABILITY PLATFORM

📖 Top Observability Platforms Compared

Tool	Best For	Open Source	Recommendation
LangFuse	Any framework, vendor-neutral	✅ Yes	★ TOP PICK
LangSmith	LangChain-heavy teams	❌ No	Great for LangChain
Arize Phoenix	ML & evaluation teams	🔍 Partial	Evaluation focus
Helicone	Cost tracking priority	❌ No	Cost-only focus
PortKey	Multiple LLM providers	🔍 Partial	Multi-provider setups

Why LangFuse?

- ✅ Open source → self-host or use cloud for free
- ✅ Vendor neutral → works with OpenAI, Anthropic, Gemini, Llama, etc.
- ✅ Full features: Traces · Metrics · Evaluations · Dashboards
- ✅ Generous free tier: 50,000 observations/month
- ✅ Concepts learned here transfer to ANY observability platform

📖 LangFuse Data Model (How Data Is Organised)

🔍 **HIERARCHY** SESSION → contains many TRACES
TRACE → single request/operation (e.g., one user message)
OBSERVATION → individual steps inside a trace

- SPAN: timed operation (retrieval, processing)
- GENERATION: LLM API call (tracks tokens + cost)
- EVENT: point-in-time occurrence (cache hit, error)

Concept	What It Represents
Session	Group of related traces (e.g., a full chat conversation)
Trace	One complete request — e.g., user sends a question
Span	Timed sub-step: retrieval, DB query, processing
Generation	LLM API call — tracks model, tokens in/out, cost, latency
Event	Instant occurrence — cache hit, error, checkpoint

📖 Why Sessions & Tags Matter for Cost Management

- 🔍 Trace Level → total cost per single request
- 🔍 Session Level → total cost per full conversation / user journey
- 🔍 Tags → cost breakdown by feature, user tier (free / pro), experiment variant

- 🔗 Find most expensive users or user groups instantly
- 🔗 Prove to leadership exactly where the budget goes

🔗 LangFuse API Levels (Choose Your Style)

API Level	How It Works	Best For
Decorator (@observe)	Annotate functions — auto-traces everything	Most teams (modern, clean)
Context Manager	Use `with` blocks for spans/generations	Explicit control needed
Low-Level SDK	Manual trace/span/generation creation	Maximum customisation
Drop-In Wrapper	Replace `openai.chat` with <code>langfuse.openai</code>	Quickest integration

🔗 Quick Setup

```
# 1. Install
uv add langfuse

# 2. .env file
LANGFUSE_PUBLIC_KEY=pk-...
LANGFUSE_SECRET_KEY=sk-...
LANGFUSE_HOST=https://cloud.langfuse.com

# 3. Decorator pattern (recommended)
from langfuse.decorators import observe, langfuse_context

@observe
def my_llm_function(prompt: str):
    response = openai_client.chat.completions.create(...)
    langfuse_context.update_current_trace(metadata={...})
    return response
```

🔗 SECTION 5 — INSTRUMENTING YOUR LLM APPLICATION

🔗 Tracing a RAG Pipeline End-to-End

- 🔗 **RAG Pipeline Steps (each becomes a LangFuse Span)**
1. User Query Received → Trace created
 2. Embed Query → Span (cheap, fast)
 3. Vector Search (ChromaDB) → Span with distance scores
 4. Build Context → Span (assembles retrieved chunks)
 5. Generate Response → GENERATION (LLM call — tracks tokens + cost)
 6. Return to User → Trace closed

🔗 What You Can See in LangFuse After Tracing a RAG Pipeline

- 🔗 Index performance: how long document loading and chunking takes
- 🔗 Time distribution: Generation is ~83% of total query time (expected)
- 🔗 Retrieval quality: distance scores showing relevance of retrieved chunks
- 🔗 Context size: number of tokens in assembled context (cost implication)
- 🔗 Total cost breakdown: input tokens + output tokens per request
- 🔗 Errors: exactly which step failed and why

🔗 Tracing Multiple Models (Model Comparison)

By tracing both Claude and OpenAI calls for the same prompt, you instantly see:

- 🔗 Claude took 8.69s for "explain theory of relativity" · OpenAI took 11.1s for same query
- 🔗 Token counts, cost breakdowns, and latency — all in one place
- 🔗 Which model gives better value for each type of query

🔗 PII Redaction Before Logging (Security)

🔗 **IMPORTANT** When logging prompts/responses in LangFuse you may accidentally capture:
Emails · Phone numbers · SSNs · Credit cards · IP addresses · Health data

SOLUTION: Redact PII patterns BEFORE logging. Set capture_input=False and capture_output=False for sensitive endpoints, then manually log redacted versions.

```
# PII Redaction Pattern
import re
PII_PATTERNS = {
    "email": r"[\w.-]+@[ \w.-]+\.[a-z]{2,}",
    "phone": r"\b\d{3}[-.\s]\d{3}[-.\s]\d{4}\b",
    "ssn": r"\b\d{3}-\d{2}-\d{4}\b",
}
def redact_pii(text):
    for label, pattern in PII_PATTERNS.items():
        text = re.sub(pattern, f"[REDACTED {label.upper()}]", text)
    return text
```

🔗 SECTION 6 — COST OPTIMISATION STRATEGIES

🔗 Strategy Overview

Strategy	Cost Savings	Effort	Priority
Prompt Optimisation	30–50%	LOW	#1 — Start here, it's free
Semantic Caching	30–50%	MEDIUM	#2 — Eliminate redundant LLM calls
Smart Model Routing	50–70%	MEDIUM	#3 — Route cheap for simple tasks
All Three Combined	70–85%	MEDIUM	#4 — Maximum savings

🔗 Strategy 1: Prompt Optimisation

🔗 **KEY PRINCIPLE** It's not about the AMOUNT of text — it's about the QUALITY.
Remove filler, be concise, eliminate repetition.

Version	Prompt Text
BEFORE (bloated, 89 tokens)	"You are a helpful AI assistant. Please note that in your response, as an AI assistant, make sure to be accurate and relevant."
AFTER (clean, 13 tokens)	"You are a helpful assistant. Be accurate and concise."
Reduction	85% fewer tokens → 30–50% cost reduction

Self-review questions for every prompt:

- ?Does the model actually need this instruction?
- ?Can I say the same thing in fewer words?
- ?Is this instruction repeated somewhere else in the prompt?
- ?Am I sending context the model won't use?

📌 Token Cost vs Reasoning Quality Trade-Off

📌 **TWO DIFFERENT GOALS — Don't confuse them!** Token Cost Optimisation → Reduce tokens (compress, remove filler)
Reasoning Quality Optimisation → Increase output quality (expand, decompose)

Sometimes they CONFLICT. Enterprise systems optimize for:
Accuracy FIRST · Cost SECOND (for high-value workflows)

Approach	When to Use	Token Effect	Quality Effect
Prompt Compression	Filler words, repeated instructions, verbose history	↓ Reduces	= Same or better
Query Expansion (Decomposition)	Complex analysis, multi-hop reasoning	↑ Increases	↑ Much better
Semantic Caching	Similar/repeated queries	↓ Eliminates LLM call entirely	= Same
Model Routing	Route simple tasks to cheap model	= Same	= Same

📌 **Enterprise Pattern: Adaptive Routing** User Query → Complexity Detector
→ Simple query? YES → Direct LLM (cheap model)
→ Complex query? NO → Decompose into subqueries → Multi-step reasoning → Synthesis

Query decomposition cost example:

Without: 1,500 tokens | With 4-subquery decomposition: 3,300 tokens
But: \$0.05 extra tokens vs \$50,000 wrong business decision = easy choice.

⚡ Strategy 2: Semantic Caching

Semantic caching matches MEANING, not exact text — far more powerful than exact-match cache.

Concept	Detail
"What's your return policy?"	→ Cache HIT for "How do I return something?" (91% similar)
Similarity Threshold	~0.92 (92%) — queries must be this similar to use cached answer
Cache TTL	24 hours for most use cases
Production Cache Hit Rate	30–50% of all queries served from cache → 0 LLM cost
Storage	ChromaDB (or any vector DB) with persistence
Embedding Model	all-MiniLM-L6-v2 (small, fast, 384 dimensions)

```
# Semantic Cache Logic (simplified)
def get_cached_response(query, threshold=0.92):
    query_embedding = embed(query)
    results = chroma_db.query(query_embeddings=[query_embedding], n_results=1)
```

```

if results and results["distances"][0][0] >= threshold:
    return results["documents"][0] # Cache HIT – no LLM call!
return None # Cache MISS – call LLM and store result

```

🔗 Strategy 3: Smart Model Routing

🔗 **BIG IDEA** 70–80% of requests can use the cheapest model available!
Route by task type → only upgrade when quality actually suffers.

```

# Intelligent Model Router
def route_model(task_type: str, complexity: str) -> str:
    if task_type in ["classification", "extraction", "simple_qa"]:
        return "claude-3-5-haiku" # $0.25/M tokens
    elif task_type in ["summarisation"]:
        return "gpt-4o-mini" # $0.15/M tokens
    elif task_type == "code_generation":
        return "claude-3-5-sonnet" # Test both, upgrade if needed
    else: # complex reasoning, analysis
        return "gpt-4o" # $2.50/M – only when needed

```

🔗 SECTION 7 — MONITORING, ALERTING & DEBUGGING

🔗 Alert Types & Thresholds

Alert Type	Threshold	Priority
Daily Spend Exceeded	> 120% of daily average	🔗 HIGH
Single Request Cost	> \$1.00 per request	🔗 HIGH
Error Rate	> 5%	🔗 CRITICAL
Latency (p95)	> 10 seconds	🔗 MEDIUM

🔗 Debugging With Traces — What You Can Find

- 🔗 Bottlenecks: "Retrieval takes 6.8s" — immediately visible in trace timeline
- 🔗 Excessive retrieval: "Fetching 50 chunks when 5 is enough" — see metadata
- 🔗 Hallucination source: wrong documents indexed → appearing in RAG context
- 🔗 Cost spikes: which feature / user / prompt is driving the cost
- 🔗 Error attribution: exactly which step failed with full error message

🔗 Three Essential Dashboard Views

Dashboard	Key Metrics
🔗 Cost	Total cost · Cost by model · Cost by feature · Top 10 users by cost
⚡ Performance	Latency p50/p95/p99 · Error rate · Cache hit rate · Throughput
🔗 Quality	Eval scores · Hallucination rate · User feedback · Relevancy scores

🔗 Webhook Alerting Pattern

```

# Cost Alert Webhook
import requests

```



```

HOURLY_COST = 15.50 # actual from LangFuse
THRESHOLD   = 10.00 # your budget

if HOURLY_COST > THRESHOLD:
    requests.post(WEBHOOK_URL, json={
        "text": f"🔔 Cost Alert! ${HOURLY_COST:.2f}/hour > ${THRESHOLD}",
        "alert_type": "cost_spike",
        "timestamp": datetime.now().isoformat(),
        "dashboard": "https://cloud.langfuse.com/dashboard",
    })
# Connect webhook to Slack, email, PagerDuty, etc.

```

📌 SECTION 8 — PRODUCTION PATTERNS & 30-DAY ROADMAP

📌 Your 30-Day Implementation Plan

Week	Focus	Deliverable
Week 1	📌 Foundations	First traces flowing in LangFuse for your app
Week 2	📌 Visibility	Cost dashboards live + alert webhooks configured
Week 3	📌 Optimisation	Semantic cache + model routing deployed in staging
Week 4	📌 Polish	PII redaction · Docs · Team training · Security review

✅ Production Checklist

- 📌 Traces flowing for every LLM call in production
- 📌 Cost dashboards accessible to the full team
- 📌 Alerts configured for spend thresholds and error rates
- 📌 Prompt optimisation applied — all filler removed
- 📌 Semantic cache deployed for common query patterns
- 📌 Smart model routing routing simple tasks to cheap models
- 📌 PII redaction active on all logging paths
- 📌 Team trained on reading LangFuse traces

📌 Course Recap & Key Takeaways

- **Observability = Money Saved** → 50–80% cost reduction; treat it as serious business, not optional
- **3 Cost Levers to always remember:**
 1. 📌 **Prompts** — optimise first, biggest impact
 2. ⚡ **Caching** — eliminate redundant calls
 3. 📌 **Routing** — right model for the right task
- **Trace Everything, Measure Constantly** → LLM behaviour changes over time; stay on top of it

- **Debug in Minutes, Not Hours** → With LangFuse (or similar), real-time data makes diagnosis fast and precise

□ Next Steps (Action Items)

- **Download all course code** — via GitHub repo or course resources folder
- **Set up LangFuse** — takes ~5 minutes, start today
- **Instrument your main LLM endpoint** — follow the course workflow
- **Implement ONE cost optimisation this week** — just one, not all at once

□ Closing Mindset

"Go build something efficient. Show off the savings — to your org, your team, or your own project. That's the point."

🔗 QUICK REVISION CHEATSHEET

🔗 Key Numbers to Remember

Item	Value
1 token ≈	4 characters 0.75 words
Output cost vs Input	2–5× MORE expensive than input
Prompt optimisation saves	30–50% cost (low effort)
Semantic cache saves	30–50% cost (medium effort)
Model routing saves	50–70% cost (medium effort)
All combined saves	70–85% cost
LangFuse free tier	50,000 observations/month
Typical cache hit rate	30–50% of production queries
Cache similarity threshold	~0.92 (92%)
Cache TTL	24 hours
ROI from observability	7× – 30× in first month

🔗 LangFuse Data Hierarchy (1-liner)

🔗 SESSION → TRACES → OBSERVATIONS (SPAN | GENERATION | EVENT)
A 10-message chat = 1 Session + 10 Traces + many Observations each

🔗 Golden Rules

- ★**Rule 1:** What you don't measure, you can't manage.
- ★**Rule 2:** Ask the model to be concise → always saves money.
- ★**Rule 3:** Choose the right model first — it's the highest leverage cost optimisation.

- ★**Rule 4:** Output tokens cost 2–5× more — control response length.
- ★**Rule 5:** 70–80% of requests can use the cheapest model with smart routing.
- ★**Rule 6:** Semantic caching matches meaning, not exact text.
- ★**Rule 7:** PII must be redacted before it ever reaches LangFuse logs.
- ★**Rule 8:** Good alerts save money. Bad alerts create noise.

📝 Notes by: LLM Observability & Cost Management Course · Pablo Deshon
Generated as curated study notes for easy learning and implementation.

❑ What is a Feedback Layer?

It's the mechanism that **captures human or system signals** about AI output quality — and uses those signals to continuously improve the system.

Without it → AI gives same quality output forever.
With it → AI gets sharper with every interaction.

❑ The Feedback Loop Architecture

```
User/System Input
↓
AI Agent / RAG Pipeline produces Output
↓
Feedback Layer captures signal
↓
Signal is stored, analyzed & processed
↓
System is refined (prompt / retrieval / model)
↓
Better Output next time ❑
```

❑ Types of Feedback Signals

Type	How Captured	Example
Explicit Feedback	User directly rates output	❑❑, 1–5 stars, "Was this helpful?"
Implicit Feedback	Behavior observed silently	User ignored answer & re-asked → bad signal
Correction Feedback	User edits the AI output	User rewrites AI draft → gold training data
Acceptance Feedback	User acts on the output	Clicked "Use this" → positive signal
Escalation Feedback	User went to human instead AI failed	→ negative signal

□ How Feedback Refines RAG Specifically

RAG has **3 layers** that feedback can improve:

Retrieval Layer

Query → Search vector DB → Fetch chunks → Pass to LLM

Feedback signals:

- Were the retrieved chunks relevant?
- Did the answer cite the right source?
- Was the context window wasted on irrelevant docs?

Refinements:

Problem	Fix via Feedback
Wrong chunks retrieved	Re-rank embeddings, tune similarity threshold
Missing context	Expand chunk size or add metadata filters
Outdated documents	Flag & refresh stale knowledge base entries
Wrong index searched	Route queries to correct knowledge domain

Generation Layer

Context + Query → LLM → Response

Feedback signals:

- Was the answer accurate?
- Was it too long / too short?
- Did it hallucinate?

Refinements:

Problem	Fix via Feedback
Hallucination	Add grounding checks, citation enforcement
Wrong tone	Adjust system prompt based on corrections
Incomplete answer	Tune prompt to request more thorough responses
Off-topic response	Add guardrails / topic filters

Knowledge Base Layer

Documents → Chunked → Embedded → Stored

Feedback signals:

- Which documents are never useful?
- Which topics have high failure rates?

Refinements:

- Remove low-quality documents
- Add missing knowledge gaps
- Re-chunk documents differently
- Add better metadata tags

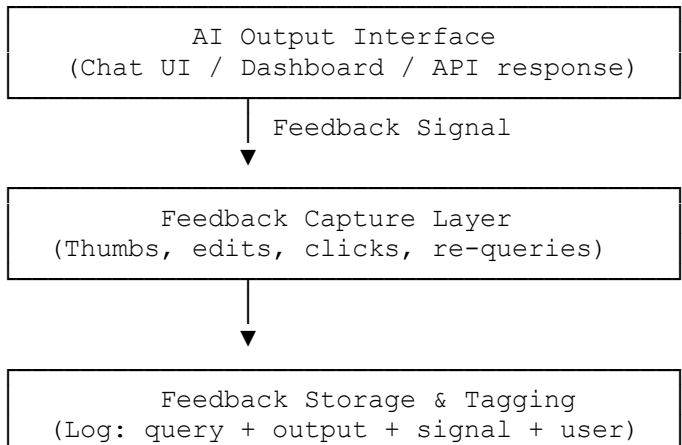
How Feedback Refines Agentic AI Specifically

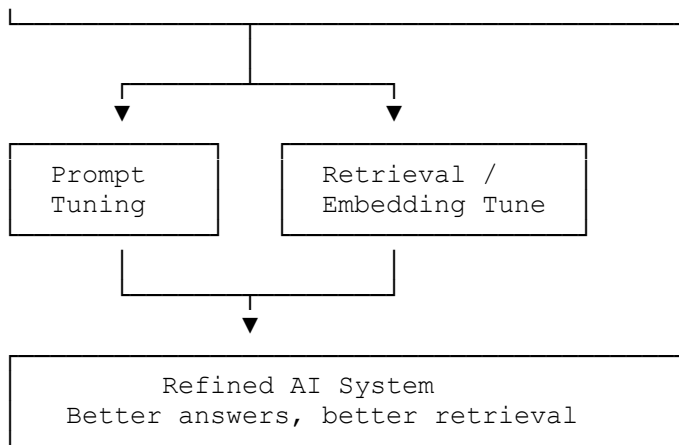
Agents are more complex — feedback can target **each agent action**:

Goal → Plan → Tool Use → Output → Feedback → Replanning

Agent Layer	Feedback Target	Refinement
Planning	Was the task broken down correctly?	Improve planning prompt
Tool Selection	Did agent pick the right tool?	Retrain tool-routing logic
Execution	Did the action produce right result?	Adjust action parameters
Memory	Did agent remember context correctly?	Fix memory retrieval logic
Final Output	Was the goal achieved?	End-to-end reward signal

Feedback Layer Architecture (Technical)





□ Feedback-Driven Improvement Strategies

Strategy	What it Does
RLHF (Reinforcement Learning from Human Feedback)	Trains model to prefer human-approved outputs
Few-shot injection	Adds good examples from feedback into prompts
Prompt versioning	A/B test prompts, keep winner based on feedback scores
Retrieval re-ranking	Use feedback to teach which chunks are most useful
Fine-tuning	Use correction data to fine-tune the base model
Confidence thresholds	Low-confidence outputs auto-flagged for human review

□ Feedback Quality Tiers

- Bronze — Thumbs up/down (volume signal, low precision)
 - Silver — User edits output (high quality training data)
 - Gold — Expert annotation (best, but expensive & slow)
-

□ Key Insight for Consulting Delivery

Build the feedback layer INTO the interface from Day 1 — not as an afterthought.

Every client-facing AI tool you deliver should have:

- ✓ A way to capture signal (even just □□)
- ✓ A log of all queries + outputs
- ✓ A review dashboard for the client team
- ✓ A scheduled refinement cycle (weekly/monthly)